

Lab 3: 启发式搜索解决15-puzzle问题

利用A*算法和ID A*算法解决15-Puzzle问题, 可自定义启发式函数。编写python程序, 展示解决15-puzzle问题的过程。

功能要求:

1. python文件要实现main函数, 该main函数的流程为: 生成或读取一个**可解的**15数码问题、列出其解法。
2. 15数码问题以空白格为移动目标, 允许的移动操作为“上下左右”4格方向。
3. main函数需要打印出当前要处理的15-puzzle, 并且按步骤以类似: "udlr..." 的形式打印解题步骤。
4. 分别实现使用A*和ID A*搜索的解题方法。

实验要求:

1. 你可以为2种算法各写一个python文件, 提交时请注意文件命名要符合可读性要求。可优化点: 将所有代码合并到一个程序中, 考虑python内置软件包argparser的使用, 允许程序在运行时读入参数, 自动选择使用那种算法; 或者参考lab2的main.py。
2. 每个算法的实验需要通过一个简单的15-puzzle来逐步验证算法的可行性, 并且对一个随机的15puzzle进行解题, 解题过程的正确性是本次实验的评分重点。
3. 你可以尝试多种启发式函数, 在实验报告中至少要描述1种启发式函数, 以及选择它的合理性。
4. 在实验报告里, 需要给出2种算法的伪代码并且粘贴关键代码及注释, 注意: 伪代码不能笼统的描述算法流程, 需要是解决15-puzzle问题的伪代码。
5. 在实验报告中需要描述调用的优先权队列(堆)的性质, 以及它在启发式搜索算法中的应用。
6. (可选) 使用适当的可视化方法(字符串打印、pygame/matplotlib等简单图形化软件包)可视化解题过程, 如果有可以将结果粘贴进实验报告。

HINTS:

1. n-数码问题的可解性证明可参考: 《15-谜问题的可达性判定》(宋文等, 电子科技大学学报 2004年10月), 本次实验不对证明做要求, 同学们有兴趣可以自行阅读。在文档中, 我们给出了生成可解的15puzzle的代码, 同学们可以直接调用。
2. 请同学们思考构造15-puzzle问题的过程与解题过程的关系(逆过程?)。
3. 随机生成的15puzzle可能状态空间巨大, 导致运行缓慢, 在实验过程可以手动构造简单的15-puzzle (8步以上), 先验证算法的正确性, 再做随机问题的实验。
4. 搜索过程中需要存储已访问的状态, 回忆python中元组的可哈希性, 以及set数据类型的使用, 对搜索环检测有帮助。
5. python有内置的优先权队列类, 使用时注意其性质。
6. 启发式函数包括但不限于: 当前未归位非数码的个数、所有数码与对应位置的曼哈顿距离等。(加分项) 在实验报告中使用数据分析不同的启发式函数对性能的影响。

生成一个可解的15-puzzle

参考代码：

```
1 import random
2 def is_solvable(puzzle):
3     # 计算逆序对的数量
4     inversions = 0
5     for i in range(16):
6         for j in range(i + 1, 16):
7             if puzzle[j] and puzzle[i] and puzzle[i] > puzzle[j]:
8                 inversions += 1
9     # 找到空格的位置
10    empty_row = 4 - (puzzle.index(0) // 4)
11    # 判断是否可解
12    return (inversions % 2 == 0) == (empty_row % 2 == 1)
13
14 def generate_random_puzzle():
15     puzzle = list(range(16))
16     random.shuffle(puzzle)
17     while not is_solvable(puzzle):
18         random.shuffle(puzzle)
19     return puzzle
20
21 # 示例使用
22 random_puzzle = generate_random_puzzle()
23 print(random_puzzle)
```

提交：

1. 将所有文件打包成一个压缩包，压缩包命名为：“学号_姓名_作业编号”，例：20240312_张三_实验2。
2. 压缩包内包含：code文件夹和实验报告PDF文件。
 - code文件夹：存放实验代码，如果有多个文件，推荐编写对应的README文件；（注意!!!）
 - PDF文件格式参考超算习堂上的模板，主要描述实现类及其类方法的过程。
3. 截止日期：**2025年4月7日晚24点**。
4. 提交地址：超算习堂